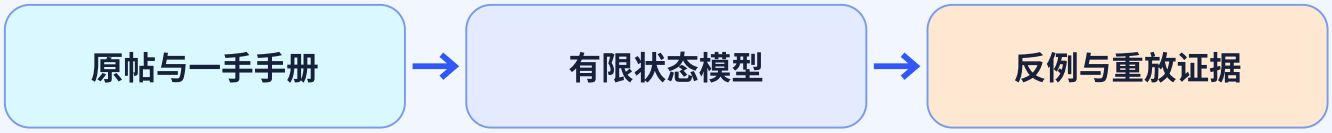
03 · 状态图 × TLA+　真实TLC / 无付费API

先画清状态，再让反例说话

一次退款插队，怎样击穿“已检查过付款”？

真实完成：反例 → 修复 → 变异再失败

官方TLC实际运行；bug退出12，修复退出0。多订单边界1/2/3分别完整检查5/25/125个状态。仅合成订单，不触碰真实支付或产品代码。



不是Agent自评，不是“任何规模 / 所有代码都已证明”

固定一手版本

pstack 0.15.36
de8ee15dd1194627d21eb5fcd41cb1873208dabb

TLC发行v1.7.4 / TLC2 2.19
源码tag:
5a47802b5c391f59ecdd44117981f4ff8c0656ba

学习范围

12问逐问SVG、5个用途、6步真实实验。

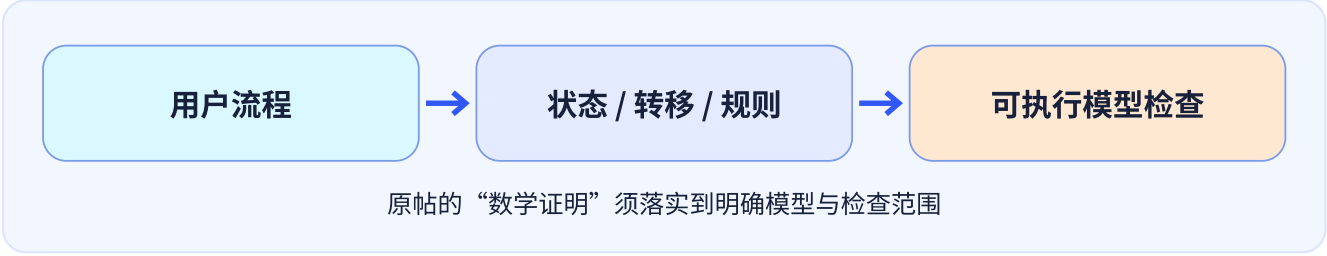
Temurin21.0.13+9仅安装在本项目录；访问2026-11-11 Asia/Shanghai。

知识库main已ls-remote确认未更新。

01 原帖真正建议做什么？状态图等于验证吗？

原帖建议先把代码和用户流程画成状态图，再用pstack的元技能生成形式化验证。引用作者给的是**假想通知系统**示例：未读线程被误标已读、勿扰时仍通知等；配图不是我们复现过的产品缺陷。

图帮助理解，规则与可执行检查才负责找反例。单看一张图，不知道分支是否遗漏、动作是否原子或数据是否过期。我们另写合成订单与TLA+，不把作者发现bug的自述当成本轮成绩；原帖及第一层引用全文可读，第二层发布帖仍截断，缺文不补写。



原帖的“数学证明”须落实到明确模型与检查范围

依据：[X0][X1][X2部分]；两张引用配图

02 pstack提供的是验证器，还是生成验证的手册？

它的`/create-formal-verification-skill`是**元技能**：选择代价高、测试不易覆盖的承诺，选择合适工具，再留下项目本地的可重跑验收说明。真正搜索状态的是TLC；涉及所有输入/任意规模时，可能需要代码级验证器或证明助手，而非强行套TLA+。

本轮一手来源是`cursor/plugins/pstack`，当前插件0.15.36。搜索先命中的backnotprop/pstack是独立镜像，已纠正归属。我们实读生成、维护与statechart手册并按方法手写实验；**没有安装/调用pstack插件或付费Agent API**，不把读提示词说成执行插件。



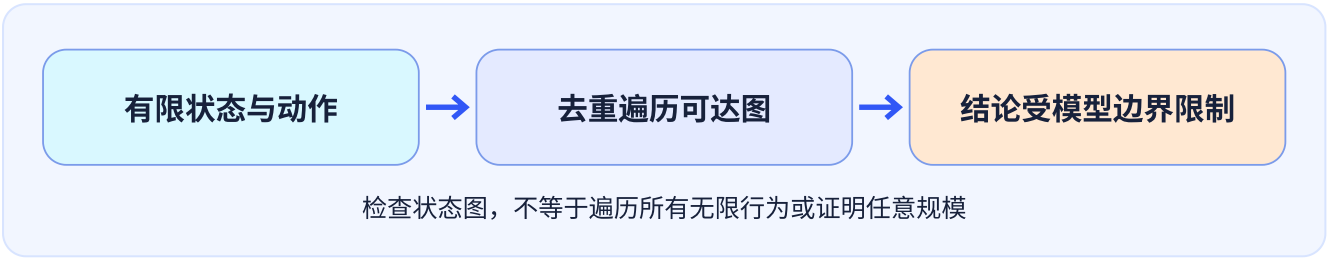
pstack手册与TLC工具是两层，不是同一个程序

依据：[P1][P2][P3]；pstack/.cursor-plugin/plugin.json

03 “所有可能走到的路”究竟指什么？

TLC从Init出发，对Next定义的转移做广度优先搜索，记录已经见过的状态。**检查完成的有限可达图**比抽样测试覆盖更多交错，但不是把无限长路径一条条枚举；一个有环的小图就能表示无限多条行为。

本例三变量的粗候选空间是2×3×2=12种组合，动作限制后固定版只有5个可达状态。通过只覆盖所给模型、初值、动作、边界与不变量；没有给真实系统、任意订单数或全部安全性质开保证。TLC使用状态指纹，日志也保留其碰撞概率估算，而非定理证明证书。

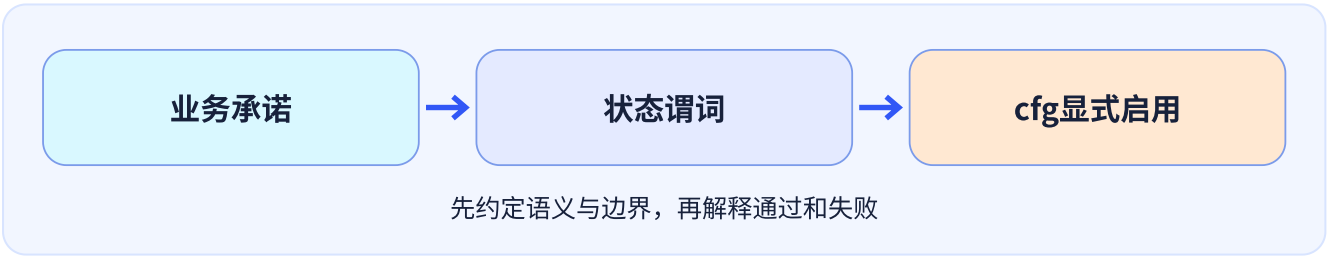


依据：[T1]；TLC真实输出；[P1]:19–29

04 先写哪条不变量？“已付款”要怎样定义？

本例把paid定义为**当前支付仍有效**，发货状态满足`NoUnpaidShipment == shipPhase = "shipped" => paid`。另有TypeOK检查布尔值和阶段取值。必须在cfg的INVARIANTS里启用，只有定义但不配置不会自动验收。

我们明确排除“发货后退款”，且此限制在bug/fixed中完全相同；否则合法售后退款也会触发该状态规则。真实业务若允许售后退款，应验证“发货当时有有效支付”的历史事实，或另建退款阶段，不能为了绿灯临时改弱原承诺。



依据：OrderCore.tla；OrderBug.cfg / OrderFixed.cfg；[P1]

05 TLA+怎样把代码步骤写清楚?

`paid = TRUE`描述当前状态；`paid' = TRUE`约束下一状态。动作是“当前守卫 ∧ 下一状态赋值”；`UNCHANGED`明确其余变量不变，`Next == Pay \/ CheckPayment \/ Refund \/ Ship…`让调度器选择一种可发生动作。

checkedPaid保存检查时的支付快照，shipPhase是new/checked/shipped。参考Python函数与每个动作一一列在verification-contract.md；把CheckPayment与Ship拆开，才允许Refund插入。这里每个函数是**合成系统的原子转移**，不声称现实数据库或支付平台也提供同样原子性。



依据：OrderCore.tla；order_reference.py；verification-contract.md

06 真实跑出的反例是什么？是几步?

支付 → 检查通过 → 退款 → 仍按旧快照发货。初始未支付；Pay使paid=true；CheckPayment令checkedPaid=true；Refund把paid变回false；ShipBug只看旧快照，最终shipped且paid=false，违反不变量。

TLC真实退出12，输出5个状态、4个转移；单工作线程默认BFS给出该模型中的最短反例前缀。日志还显示生成8次、发现6个不同状态、队列余1；6不是反例长度，余1说明本次在违规处提前停止，不是完整枚举成功。



依据：results/bug-stdout.txt；results/bug-summary.json；replay.json

07 怎么修复，又怎样避免修复只是模型幻觉？

ShipFixed在同一原子动作中增加实时paid守卫，而不是只信checkedPaid缓存；其余动作、退款前提和不变量保持不变。TLC修复版真实退出0，完整探索5个可达状态，队列为0；退款后的已检查状态只能先重新支付，不能直接发货。

我们把反例逐步重放到自编Python参考实现：bug模式确实达到违规状态，fixed模式阻断最后一步；又核对固定版TLC DOT的**全部状态集合**与Python枚举相同。这是合成实现的交叉证据，不是现实产品的精化证明；若真正发货发生在事务之外，仍可能有新的检查—使用竞态。



依据：ShipFixed；results/fixed-summary.json；results/replay.json

08 为什么终态要有Done，而不直接关掉死锁检查？

在TLC眼中，“没有后继”是死锁，哪怕业务认为订单已经结束。我们只在shipped终态加`Done == shipPhase = "shipped" /\ UNCHANGED vars`，表示完成后可以保持原状态；其他阶段仍必须有可走动作。

没有使用`-deadlock`或CHECK_DEADLOCK FALSE去掩盖问题。终态自环不增加新状态，且不是“保证最终发货”；它只是把预期结束和意外卡住区分开。本轮未故意注入独立死锁变异，因此不把死锁检测的全部分支说成已实测。



依据：OrderCore.tla:Done；OrderMany.tla:Done；[P3]:25–27

09 生成状态、不同状态、订单规模怎么读?

generated包含重复到达；distinct是去重后的状态数；queue=0且正常完成，才支持对本配置可达图的通过结论。单订单修复版8/5/0；两订单62/25/0；三订单452/125/0，三组均真实退出0。

独立订单每单5个局部状态，因此本模型有5^N个组合；这解释了5、25、125，不是性能跑分或对真实系统规模的预测。多订单没有共享库存、并发限额或资金总额约束，也未开SYMMETRY/VIEW/CONSTRAINT缩减；不能据这些数字宣布任意N都已检查。



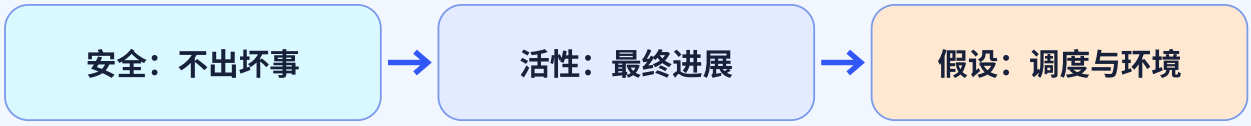
数字来自TLC真实输出；独立组合不是现实负载基准

依据：results/orders1/2/3-summary.json；OrderMany.tla

10 规则永不破坏，能推出业务最终完成吗?

不能。“未有效支付不发货”是**安全性**：坏事不发生；“最终会发货”是**活性**：好事终会发生。即使安全规则成立，系统仍可反复支付/退款或一直停留，最终发货未必成立。

本轮没有配置PROPERTY活性公式或公平性假设，不声称活性通过。时间、崩溃恢复、重复消息、金额、外部支付回执与实际发货接口均未建模。若新增公平性，必须说明调度/业务为什么真能满足，不能靠加一个假设把不进展消掉。



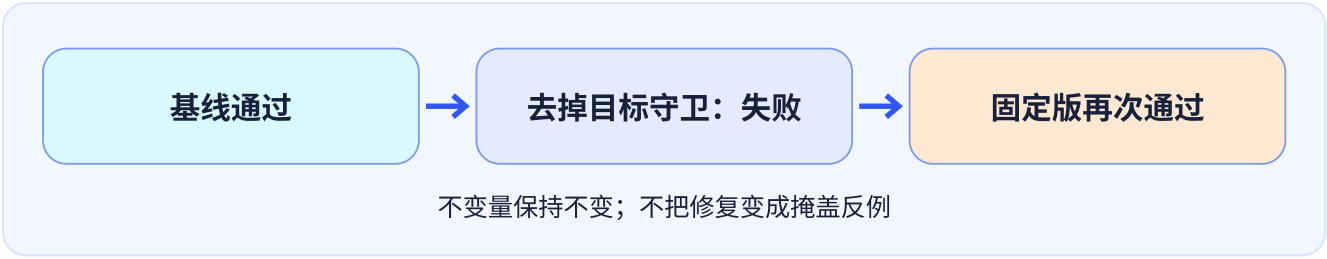
通过的是已启用的安全不变量，不是整个业务正确性

依据：[P1]:19、26；两份cfg；本轮范围声明

11 怎样证明检查不是永远给绿灯的装饰?

把固定版的ShipFixed改回缺实时paid守卫的ShipBug，保留同样INVARIANTS和退款前提。该模型变异真实再次退出12，产生同一条四步反例；说明这套检查确实对目标缺陷敏感。之后再次跑未改动的固定版，仍是5状态、退出0。

这是模型层变异与合成参考代码重放，不是对任何现有订单产品做变更。不要删掉不变量、禁止退款或缩小到看不见bug的范围来“修复”。上游要求“让检查能失败”，其证据必须是实际红灯与反例，而不是Agent说自己认真检查了。

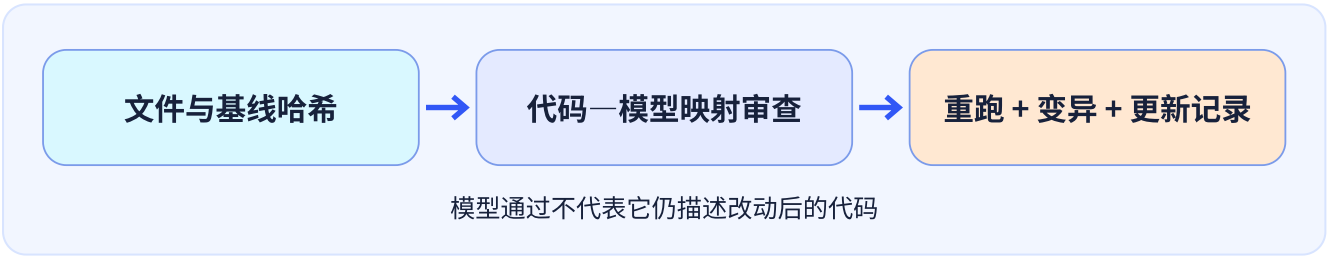


依据：results/mutant-stdout.txt；固定版复跑日志；[P1]:23–39

12 代码变了，怎样知道旧模型已经过期?

pstack维护手册区分in sync、updated、bug、blocked：先读覆盖文件自同步点以来的变化，再判断是模型漂移、承诺有意改变，还是实现违反规则。更新必须来自代码结构与业务证据，不是来自“怎样让结果变绿”。

本练习是无Git提交的scratch，所以用coverage-manifest.json固定合成实现/模型的SHA256，**不冒造sync commit**，也不把pstack仓库SHA当订单代码版本。verification-contract.md记录动作映射、命令、边界与复跑方法；它是候选验收合同，未注册全局技能，也没有所谓独立审核。



依据：[P2]；coverage-manifest.json；verification-contract.md

5个用途：高风险承诺，比大而全更重要

01 · 订单 / 退款交错

把检查支付、退款、发货拆成真实原子边界，找出旧快照继续被使用的路径；别把产品售后退款语义排除后忘记说明。

02 · 权限撤销与动作提交

建模授权检查到提交之间是否可能被撤权；这里仅是应用建议，本轮没有控制真实权限或改安全设置。

03 · 任务租约与重复执行

围绕“同一任务不被两个worker同时执行”建模过期、续租与重试；不能从本订单实验直接推出恰好一次执行。

04 · 通知与已读状态

把图里“打开频道不等于读到线程”的业务判断变成状态谓词，再把反例翻译成用户可理解的交互顺序。

05 · 维护长期验收资产

保留覆盖文件、假设、同步基线、可复跑命令与能失败的变异；代码结构改了先审模型是否失真。

查重：知识库已有LangGraph状态图、证据状态机、动作投递与安全重试。本项不重复它们；新增TLC有限图检查、反例敏感性与模型维护合同。

6步主实验 · 真正的红灯与绿灯

01 固定来源与隔离工具

目的：确认知识库未变、使用官方发布工具并校验。

```
cd /workspace/learn-batch/03-tla-statemachine
git ls-remote https://github.com/Smashwinny/reminder_knowlege refs/heads/main
python3 exercise/setup_tools.py
```

真实结果：知识库仍为a867477…；Java21.0.13+9启动成功；TLC SHA256与固定锁值匹配、SHA1与发布正文匹配。

验证：sources/toolchain.json记录版本、URL、SHA；JRE只在tools中，未改系统设置。脚本为初次真实下载/解压的可复现版本，本轮再次执行为缓存校验。

02 写清状态、原子步骤与同一规则

目的：生成bug/fixed模型及参考代码；不通过削弱规则修复。

```
cd /workspace/learn-batch/03-tla-statemachine
python3 _tools/create_exercise.py
python3 -m py_compile exercise/order_reference.py exercise/run_labs.py
```

真实结果：模型与驱动真实落盘；Python语法检查退出0；两版cfg启用完全相同的TypeOK和NoUnpaidShipment。

验证：查看OrderCore.tla与verification-contract.md的动作映射。TLA+语法/语义由随后真实TLC调用解析，不把Python编译当TLA校验。

完整命令、stdout/stderr与实际退出码在experiment-log.md及exercise/results。未调用真实支付接口、付费API或全局pstack插件。

6步主实验 · 真正的红灯与绿灯

03 真实搜索并保留违规轨迹

目的：用TLC而非手算或Agent判断，找到旧支付快照的反例。

```
cd /workspace/learn-batch/03-tla-statemachine/exercise
python3 run_labs.py bug
```

真实结果：TLC原始OS退出12；NoUnpaidShipment违反；5状态/4步轨迹，生成8、不同6、队列余1。

验证：results/bug-stdout.txt与bug-summary.json。驱动确认预期失败后退出0；不是把TLC失败抹成模型通过。完整Java命令另存*-command.json。

04 修复同一原子动作，完成检查

目的：增加实时paid守卫，保持退款前提和不变量不变。

```
cd /workspace/learn-batch/03-tla-statemachine/exercise
python3 run_labs.py fixed
```

真实结果：TLC退出0；生成8、不同5、队列0，完整模型检查无错误。

验证：results/fixed-stdout.txt和fixed-graph.dot；不要把“5个状态”写成5条测试或任意规模证明。

完整命令、stdout/stderr与实际退出码在experiment-log.md及exercise/results。未调用真实支付接口、付费API或全局pstack插件。

6步主实验 · 真正的红灯与绿灯

05 扩大有限边界并确认检查会失败

目的：独立订单N=1/2/3分别运行；模型变异去掉目标守卫。

```
cd /workspace/learn-batch/03-tla-statemachine/exercise
python3 run_labs.py scaling
python3 run_labs.py mutation
```

真实结果：边界检查分别5/25/125个不同状态，退出全0；变异再次退出12，四步反例重现。

验证：orders1/2/3-summary.json与mutant-stdout.txt；没有禁用不变量、改变退款前提或隐藏出错状态。

06 重放到合成代码并确认固定版仍通过

目的：逐步回放真实TLC轨迹，交叉检查状态集合，再复跑固定版。

```
cd /workspace/learn-batch/03-tla-statemachine/exercise
python3 run_labs.py replay
python3 run_labs.py fixed
```

真实结果：Python bug路径确实违规，fixed阻断末步；固定版DOT全部状态集合与Python5状态相同；最终固定版仍退出0。

验证：results/replay.json、coverage-manifest.json和最终fixed日志；参考实现是合成程序，不声称对某个真实订单产品做了代码级证明。

完整命令、stdout/stderr与实际退出码在experiment-log.md及exercise/results。未调用真实支付接口、付费API或全局pstack插件。

证据分层：真正完成了什么？

层级	真实完成	仍未覆盖
来源	原帖/第一引用全文；第二发布帖部分；官方技能固定SHA	第二发布帖缺文与作者实际产品bug数量未核实
TLC工具	官方release、Java隔离安装、校验、真实执行	不是源码重编译或供应链签名审计
合成订单	反例、修复、1/2/3订单、模型变异、Python重放	非真实支付/数据库/并发产品；非精化证明
交付	代码/配置/原始输出/候选验收合同	未安装/调用pstack插件，不冒称独立审核

待验证：真实业务代码的原子边界与模型对应、外部支付/发货副作用、售后退款、崩溃/乱序/重试、活性与公平性、任意订单规模。学习实验已真实完成；这些属于明确留出的外部适用性，不把它们涂成通过。

一手来源

[X0] https://x.com/i/status/2112783187667734833

[X1] https://x.com/poteto/status/2112736749281759566

[X2] https://x.com/poteto/status/2111926622278521253

[P] https://github.com/cursor/plugins/tree/de8ee15dd1194627d21eb5fcd41cb1873208dabb/pstack

[T1] https://docs.tlapl.us/using:tlc:start

[T2] https://github.com/tlaplus/tlaplus/releases/tag/v1.7.4

P1=create-formal-verification-skill；P2=maintain-formal-verification-skill；P3=statechart。完整URL、校验与可读状态见sources.md。访问2026-11-11 Asia/Shanghai。